

Render 를 활용한 배포 가이드 (FastAPI)

이 문서는 **Render의 Hobby(무료) 플랜**을 기준으로 작성되었습니다.

배포 방식은 **GitHub 저장소 연동 방식**만 다룹니다. (Render CLI, Docker 등은 다루지 않습니다.)

이 가이드에서 다루는 범위(실습용 배포)에서는 **카드 등록 없이 무료로 충분히 진행 가능**합니다.

목차

목차

1. Render란?

2. 비용 안내

3. 사전 준비물

4. 배포하기

Step 1. Render 대시보드에서 새 Web Service 생성

Step 2. GitHub 저장소 연결

Step 3. 배포 설정 입력하기

Step 4. 인스턴스 타입 선택

Step 5. 환경 변수 설정 및 배포

5. 유의할 점

 스핀 다운(Spin Down) — 가장 자주 헛갈리는 부분

 환경변수 관리

6. 팁

7. FastAPI 배포 시 자주 발생하는 문제

8. 용어 정리

1. Render란?

Render는 **PaaS(Platform as a Service, 서비스형 플랫폼)**입니다. 조금 더 쉽게 설명하면, "서버를 직접 사서 세팅하지 않고, 이미 만들어진 서버 환경을 빌려서 내 코드만 올려 실행시키는 서비스"라고 이해하시면 됩니다.

비유하자면, 직접 건물을 짓고 전기·수도 공사를 하는 대신, 이미 다 지어진 사무실 공간에 내 짐(코드)만 들고 들어가서 바로 일을 시작하는 것과 비슷합니다.

Render의 가장 큰 장점은

"GitHub 저장소와 연동해두면, 코드를 push할 때마다 자동으로 빌드하고 배포해준다."는 점입니다.

즉, 한 번 연결해두면 그 다음부터는 "코드 수정 → GitHub에 push → 자동 배포"라는 흐름으로 반복 작업 없이 서비스를 업데이트할 수 있습니다.

2. 비용 안내

비용 요약 (Hobby 플랜 기준)

항목	비용
워크스페이스 (Hobby 플랜)	무료
웹 서비스 (Free 인스턴스, 512MB RAM)	무료 (신용카드 등록 불필요)
월 대역폭(트래픽) 한도	5GB까지 무료, 초과 시 GB당 \$0.15 과금
유료 전환 시 (Starter 인스턴스)	\$7/월부터

⚠ 다만 Free 인스턴스는 **15분간 요청이 없으면 자동으로 꺼지는** 특성이 있습니다.

⚠ 월 5GB 대역폭 한도는 2026년 4월 요금제 개편으로 새로 생긴 제한이므로 실습 중 트래픽이 많은 테스트(대용량 파일 반복 업/다운로드 등)를 할 계획이라면 참고해두는 것이 좋습니다. 기본적인 기능 테스트는 Local에서 진행하는 것을 추천합니다.

3. 사전 준비물

- ☐ GitHub 계정 및 FastAPI 프로젝트가 올라간 저장소
- ☐ Render 계정 (GitHub 계정으로 바로 가입 가능)
- ☐ git hub 프로젝트 루트에 `requirements.txt` 파일 (설치할 패키지 목록)

저장소(Repository)에 접근하면 파일 목록에서 app 진입점과 requirements.txt 파일을 바로 확인할 수 있어야 합니다.



FastAPI 프로젝트 기본구조 (참고)

my-fastapi-app/

├─ main.py (또는 app/)

├─ requirements.txt

├─ ...

└─ .gitignore

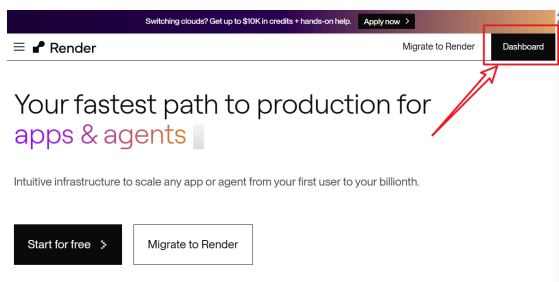
← FastAPI 앱 진입점

← 패키지 목록 (반드시 필요!)

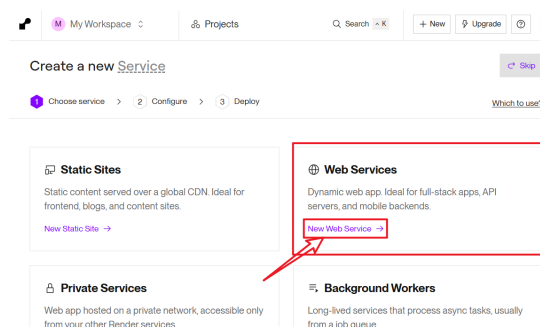
← .venv, __pycache__ 등 제외

4. 배포하기 Step 1. Render 대시보드에서 새 Web Service 생성

- Render에 로그인 후, 대시보드에서 **New** → **Web Service**를 클릭합니다.



우측 상단 Dashboard 클릭



Web Services 선택

Step 2. GitHub 저장소 연결

GitHub 계정을 연결하고, 배포할 저장소를 선택합니다. 만약 목록에 저장소가 보이지 않는다면 **Configure GitHub App**을 눌러 Render에 저장소 접근 권한을 추가로 부여해야 합니다.

Configure and deploy your new Web Service

Choose service > 2 Configure > 3 Deploy

Need help? Docs

Source Code

Git Provider Public Git Repository Existing Image

No repositories found

Connect your Git provider to deploy from your existing repositories.



Configure your Git provider to give Render permission to access your repositories.



git hub 선택

Install Render

Install on your personal account eduedy

for these repositories:

☒ All repositories
This applies to all current and future repositories owned by the resource owner. Also includes public repositories (read-only).

☐ Only select repositories
Select at least one repository. Also includes public repositories (read-only).

with these permissions:

☒ Read access to Dependabot alerts, code, and metadata

☒ Read and write access to actions, checks, commit statuses, deployments, environments, issues, pull requests, repository hooks, and workflows

User permissions
Render can also request users' permission to the following resources. These permissions will be requested and authorized on an individual-user basis.

☒ Read access to email addresses

Next: you'll be directed to the GitHub App's site to complete setup.

Render와 연동 승인

Configure and deploy your new Web Service

Choose service > 2 Configure > 3 Deploy

Need help? Docs

Source Code

Git Provider Public Git Repository Existing Image

Search

Credentials (1)

/ ai-onboarding-team-project

/ ai-onboarding-single-project

배포하고자 하는 Git 저장소를 클릭

배포할 저장소(Repository) 선택

Step 3. 배포 설정 입력하기

- Language: Python 3 선택
- Build Command: `pip install -r requirements.txt`
- Start Command: `uvicorn main:app --host 0.0.0.0 --port $PORT`

⚠️ `--host 0.0.0.0` 과 `--port $PORT` 는 반드시 넣어야 합니다. 이 옵션이 없으면 Render 가 내 서버에 접속할 방법을 찾지 못해 배포가 실패합니다. (`main` 은 실행 파일명, `app` 은 FastAPI 인스턴스 변수명이므로 본인 프로젝트 구조에 맞게 수정하세요.)

Source Code

eduardo / ai-onboarding-team-project · 17h ago

Edit

Name

A unique name for your web service.

ai-onboarding-team-project

Language

Choose the runtime environment for this service.

Python 3 Python 선택

Branch

The Git branch to build and deploy.

master

Region

Your services in the same region can communicate over a private network.

Ohio (US East)

Region

Your services in the same region can communicate over a private network.

Ohio (US East)

Root Directory

Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a monorepo.

e.g. src

Build Command

Render runs this command to build your app before each deploy.

\$ pip install -r requirements.txt

Start Command

Render runs this command to start your app with each deploy.

\$ uvicorn main:app --host 0.0.0.0 --port \$PORT

Language 설정 화면

Build / Start Command 설정 화면

Step 4. 인스턴스 타입 선택

- Free를 선택합니다.

Instance Type

For hobby projects

Free

512 MB (RAM)

\$0 / month

0.1 CPU

⚠️ Upgrade to enable more features

Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

For professional use

For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

Starter

512 MB (RAM)

Standard

2 GB (RAM)

Instance Type 선택 화면

Render 를 활용한 배포 가이드 (FastAPI)

5

Step 5. 환경 변수 설정 및 배포

- 필요한 환경 변수를 설정합니다. (예: `OPENAI_API_KEY`, `PYTHON_VERSION`, ...)

⚠ 현재 Render는 Python 3.14.3 버전이 기본으로 적용되어 있어 pip install 중 에러 발생이 높습니다.

환경 변수에서 `PYTHON_VERSION` / `3.11` 로 설정해주세요.

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

PYTHON_VERSION	3.11	🗑
----------------	------	---

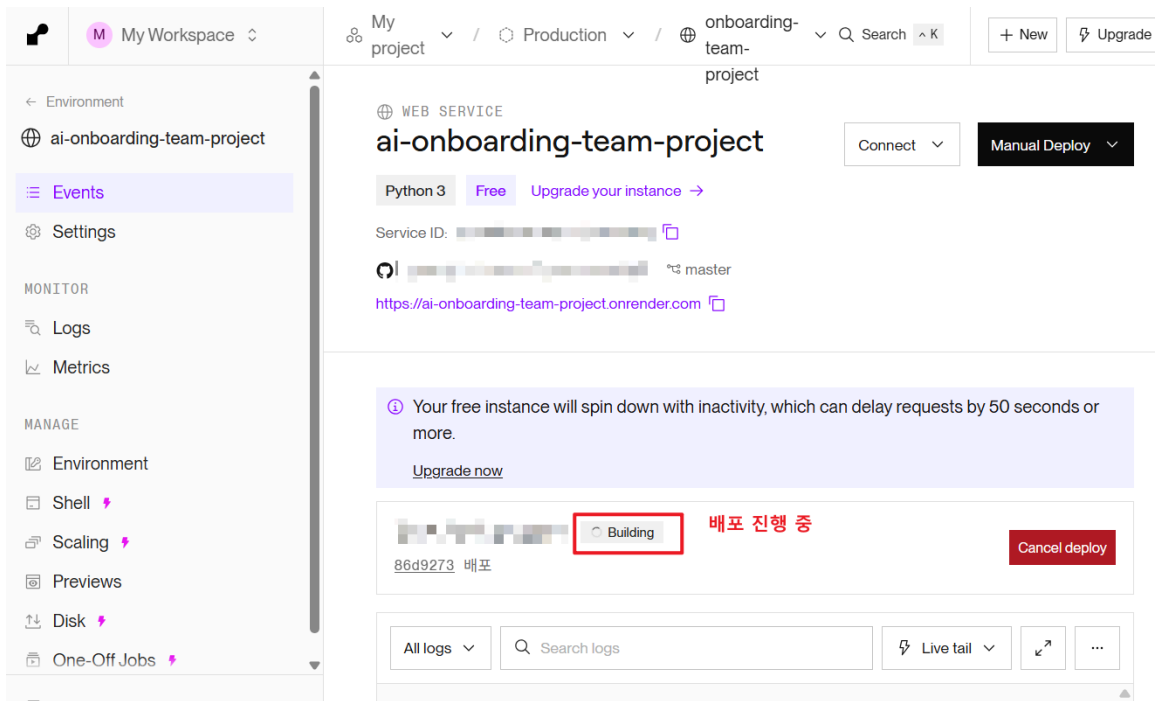
+ Add Environment Variable 📄 Add from .env Python 3.11을 사용할 수 있도록 환경변수 등록

> **Advanced** 모든 설정이 끝나면 배포 버튼 클릭

Deploy Web Service

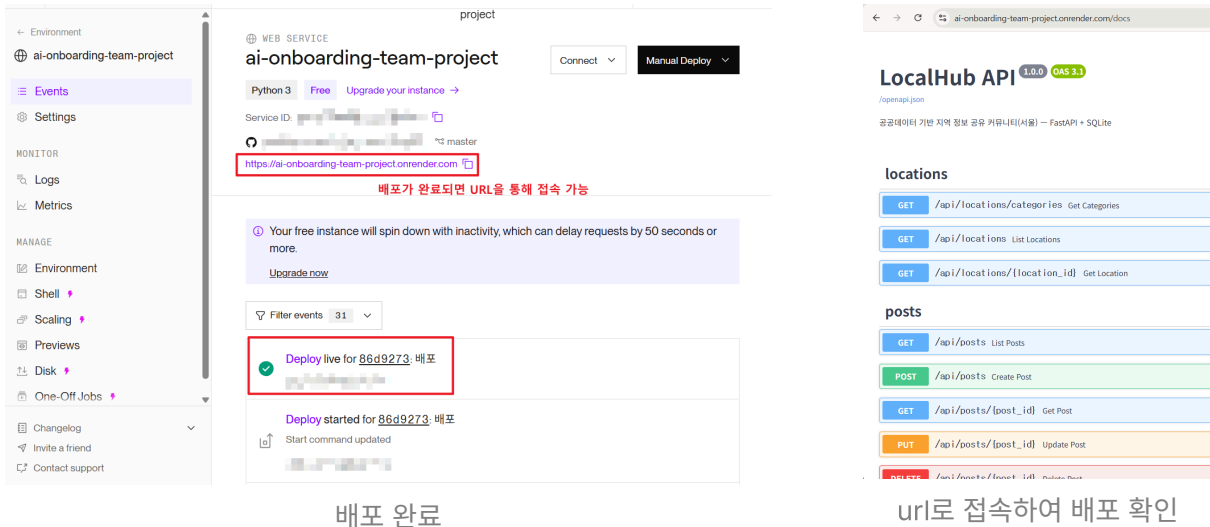
Python Version 환경 변수 설정

- Deploy Web Service** 버튼을 누르면 빌드와 배포가 자동으로 시작됩니다.



배포 진행 상황 확인

- 배포가 완료되면 `https://내서비스이름.onrender.com` 형태의 URL이 생성됩니다. 이 주소 뒤에 `/docs` 를 붙여 접속하면, FastAPI가 자동으로 만들어주는 Swagger UI(API 테스트 화면)가 뜨는지 확인해보세요.



배포 완료

url로 접속하여 배포 확인

여기까지 정상적으로 보인다면 배포 성공입니다! 🎉

5. 유의할 점 ⚡ 스핀 다운(Spin Down) — 가장 자주 헛갈리는 부분

Free 인스턴스는 **15분간 외부 요청이 없으면 자동으로 종료(스핀 다운)**됩니다.

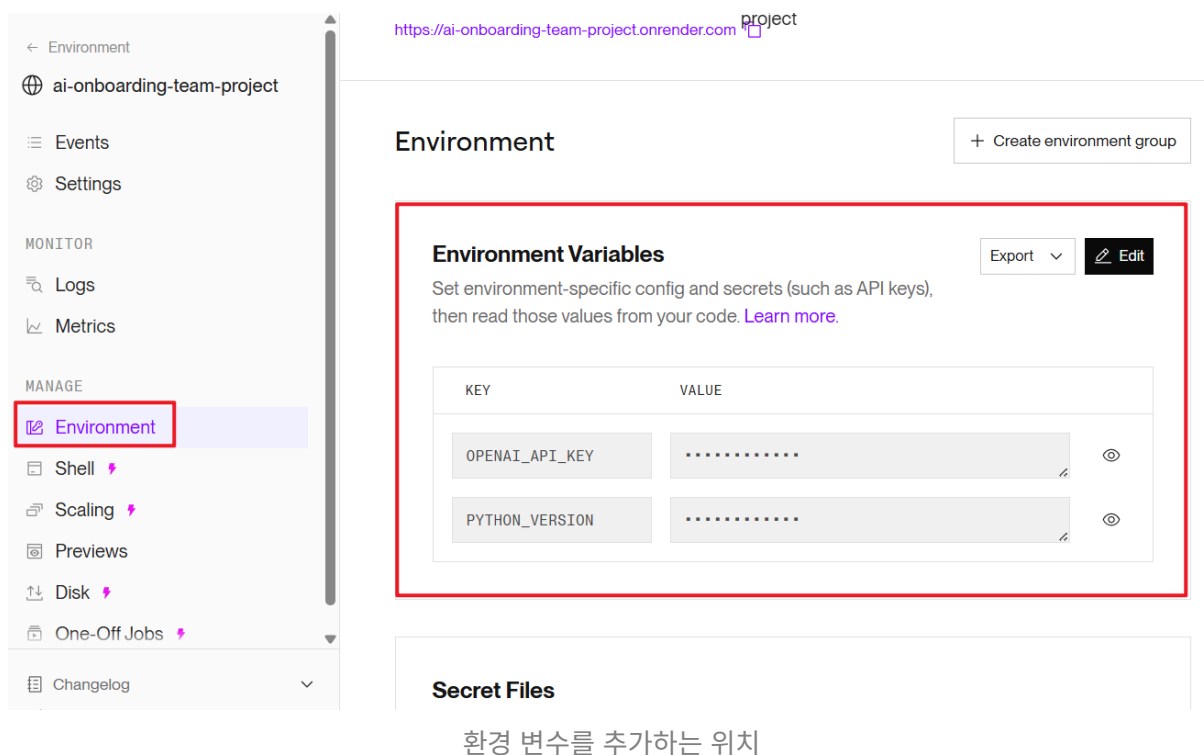
이후 다시 요청이 들어오면 서버가 재기동(스핀 백업)되는데, 이 과정에서 **최대 약 1분 정도**

응답이 느려질 수 있습니다.

- 실습 중 갑자기 응답이 오래 걸린다고 해서 배포가 잘못된 것이 아닙니다. 스핀 다운 이후 첫 요청이라 그런 것일 수 있으니 당황하지 마세요.
- 팀원에게 데모 링크를 공유하기 전에는, 미리 한 번 접속해서 서버를 깨워두는 것을 권장합니다.
- 이 문서 범위(실습·프로토타입)에서는 큰 문제가 되지 않지만, 상시 응답이 필요한 경우라면 유료 인스턴스로 전환해야 합니다.

환경변수 관리

`.env` 파일은 절대 GitHub에 올리지 마세요. 반드시 `.gitignore`에 포함시키고, 민감한 값(API 키 등)은 Render 대시보드의 **Environment** 탭에서 등록합니다.



The screenshot shows the Render dashboard for a project named 'ai-onboarding-team-project'. The left sidebar has a 'MANAGE' section with 'Environment' highlighted. The main content area is titled 'Environment' and contains a red-bordered box labeled 'Environment Variables'. Inside this box, there's a description and a table with two rows: 'OPENAI_API_KEY' and 'PYTHON_VERSION', both with masked values. To the right of the table are 'Export' and 'Edit' buttons. Below the red box is a 'Secret Files' section with the text '환경 변수를 추가하는 위치'.

KEY	VALUE
OPENAI_API_KEY	
PYTHON_VERSION	

6. 팁

- **Health Check 경로 설정:** 서비스가 정상 동작 중인지 Render가 주기적으로 확인할 수 있도록 헬스 체크용 엔드포인트(예: `GET /health`)를 만들어두면 좋습니다.

- **로그 확인:** 배포 후 문제가 생기면 가장 먼저 대시보드의 **Logs** 탭을 확인하세요. 에러 메시지 대부분이 여기에 출력됩니다.
- **Manual Deploy vs Auto Deploy:** 특정 커밋만 골라서 배포하고 싶을 때는 대시보드에서 **Manual Deploy**를 사용할 수 있습니다.
- **Instant Rollback:** 새 배포에 문제가 생겼다면, 대시보드에서 이전 정상 배포 버전으로 즉시 되돌릴 수 있습니다.



7. FastAPI 배포 시 자주 발생하는 문제

증상	원인	해결 방법
<code>ImportError: attempted relative import with no known parent package</code>	상대경로 import 사용	절대경로 import로 변경하거나, 폴더에 <code>__init__.py</code> 추가
배포 후 접속 시 <code>Bad Gateway</code>	Start Command에 <code>--host 0.0.0.0 -port \$PORT</code> 가 빠짐	Start Command를 <code>uvicorn main:app --host 0.0.0.0 --port \$PORT</code> 형태로 수정
빌드 실패 (패키지 설치 오류)	<code>requirements.txt</code> 누락 또는 버전 충돌	로컬에서 <code>pip freeze > requirements.txt</code> 로 정확한 버전 목록 재생성
프론트엔드에서 API 호출 시 CORS 에러	FastAPI에 CORS 설정이 없음	<code>fastapi.middleware.cors.CORSMiddleware</code> 추가
한동안 안 쓰다가 접속하니 응답이 느림/타임아웃처럼 보임	스핀 다운 이후 첫 요청	배포 문제가 아님. 최대 1분 정도 기다린 후 재시도

8. 용어 정리

용어	설명
PaaS (Platform as a Service)	서버 인프라를 직접 관리하지 않고, 코드만 올리면 실행 환경을 대신 제공해주는 서비스 형태
CI/CD	코드를 저장소에 올리면 자동으로 빌드·테스트·배포까지 이어지는 자동화 흐름
빌드 커맨드 (Build Command)	배포 전에 실행되는 명령어. 주로 필요한 패키지를 설치하는 데 사용 (예: <code>pip install -r requirements.txt</code>)
스타트 커맨드 (Start Command)	서버를 실제로 실행시키는 명령어 (예: <code>uvicorn main:app ...</code>)
ASGI / WSGI	파이썬 웹 서버가 요청을 처리하는 방식의 규격. WSGI는 한 번에 하나씩 순차 처리, ASGI는 비동기로 동시에 여러 요청 처리 가능 (FastAPI는 ASGI 기반)
스핀 다운 / 스핀 백업 (Spin Down / Spin Up)	무료 인스턴스가 일정 시간 요청이 없으면 자동으로 꺼졌다가(스핀 다운), 다음 요청이 오면 다시 켜지는(스핀 백업) 동작
환경변수 (Environment Variable)	코드에 직접 적지 않고 외부에서 주입하는 설정값. API 키, 비밀번호 등 민감한 정보를 코드와 분리해서 관리할 때 사용
워크스페이스 (Workspace)	Render에서 팀/개인 단위로 프로젝트들을 묶어서 관리하는 단위
인스턴스 타입 (Instance Type)	개별 서비스(웹서비스, DB 등)에 부여하는 서버 사양 등급(Free, Starter, Standard 등). 워크스페이스 요금제와는 별개로 서비스마다 선택